Johan Manuel Vicente Berroa Curso de Docker

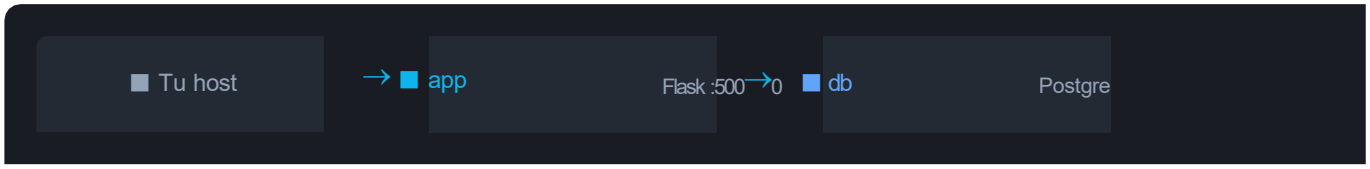
Práctica Guiada de Docker

App Flask + PostgreSQL · Red personalizada · Volumen persistente

Esta práctica te guía paso a paso para levantar manualmente, sin docker-compose, una aplicación Python (Flask) conectada a una base de datos PostgreSQL, usando una red personalizada y un volumen persistente.

■ Tecnologías	Docker Engine · Python 3.12 · Flask 3 · PostgreSQL 16
■ Red	docker network create --driver bridge lab-net
■ Volumen	docker volume create pgdata
■ App	Imagen personalizada construida con docker build
■ Acceso	Desde el host en http://localhost:5000
■ Duración	~30–45 minutos

ARQUITECTURA



1 Crear la red y el volumen

Infraestructura base

Antes de correr cualquier contenedor creamos la red personalizada y el volumen de datos. La red permite que los contenedores se comuniquen por nombre (db en vez de una IP dinámica). El volumen garantiza que los datos de PostgreSQL sobrevivan si el contenedor se reinicia o borra.

Crear la red

```
# Crear red tipo bridge llamada 'lab-net'
docker network create --driver bridge lab-net
a3f8b1c2d4e5f6a7b8c9d0e1f2a3b4c5d6e7f8a9
```

Crear el volumen

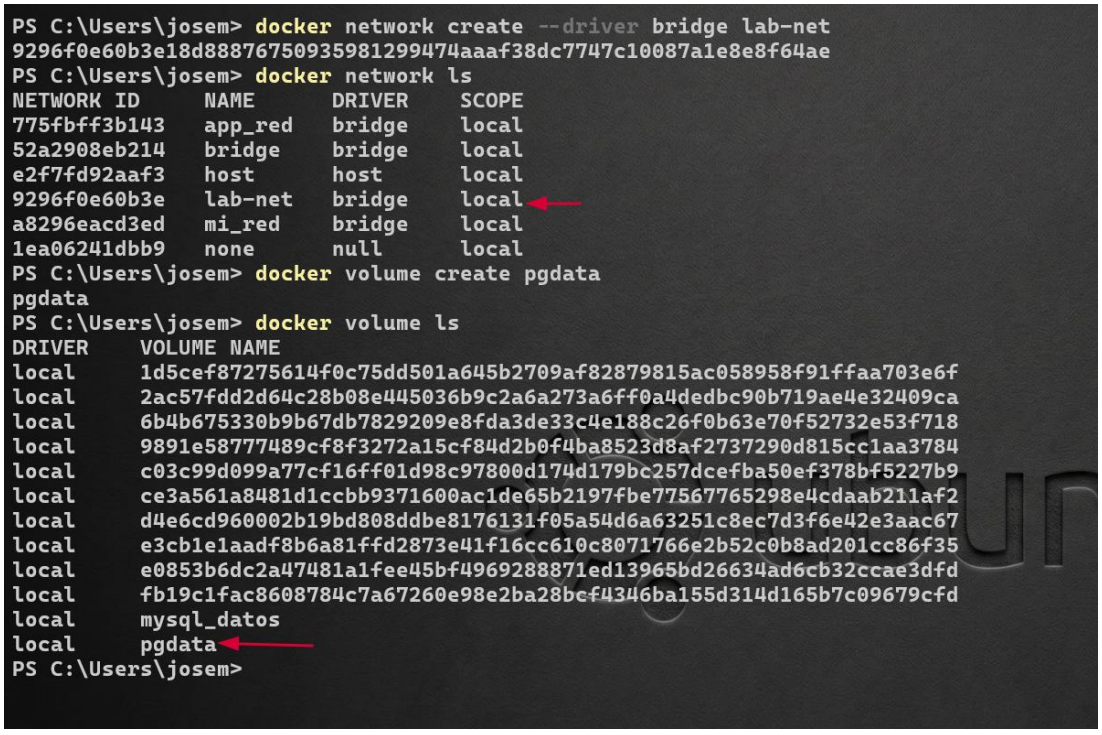
```
# Crear volumen para persistir datos de PostgreSQL
docker volume create pgdata
pgdata
```

■ La red lab-net permite resolución de nombres: el contenedor 'app' podrá conectarse a 'db' simplemente usando ese nombre como hostname, sin IPs.

VERIFICACIÓN

verificar creación

```
docker network ls
NETWORK ID     NAME        DRIVER    SCOPE
xxxxxxxxxxxx   lab-net     bridge    local   ✓
docker volume Ls
DRIVER    VOLUME NAME
local     pgdata   ✓
```



2 Levantar PostgreSQL

Contenedor de base de datos

Corremos PostgreSQL 16 Alpine (imagen liviana) conectado a nuestra red y con el volumen montado. El flag --name db es clave: así Flask lo encontrará por ese nombre en la red.

docker run — PostgreSQL

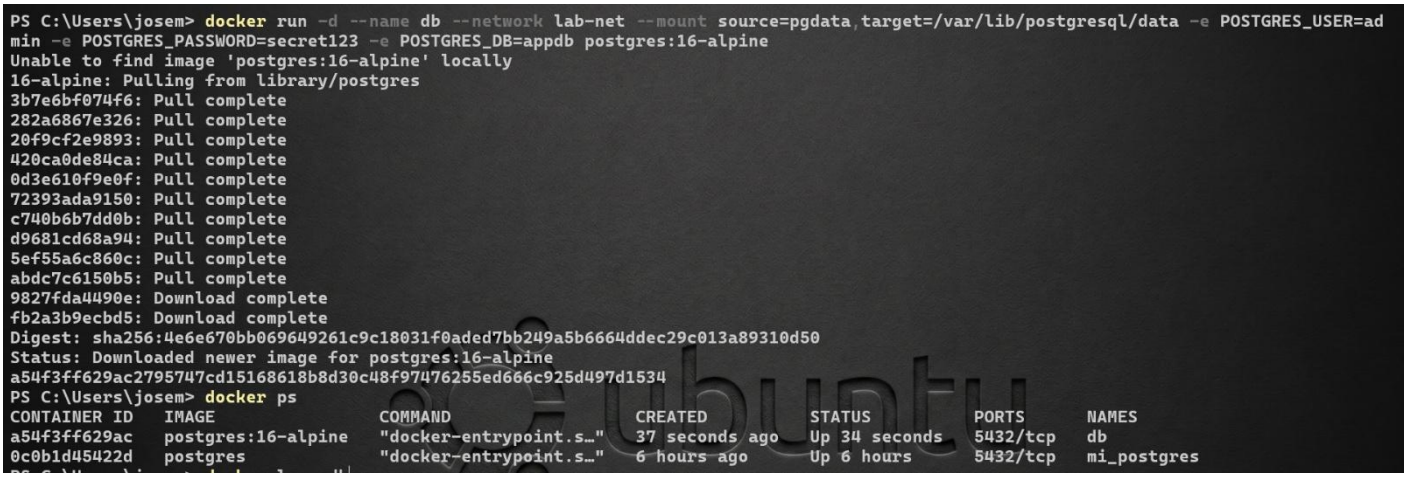
```
docker run -d \
     --name db \
     --network lab-net \
     --mount source=pgdata,target=/var/lib/postgresql/data \
     -e POSTGRES_USER=admin \
     -e POSTGRES_PASSWORD=secret123 \
     -e POSTGRES_DB=appdb \
     postgres:16-alpine
b7e3f2a1c4d5e6f7a8b9c0d1e2f3a4b5c6d7e8f9
```

`-d`	Corre en background (detached mode)
`--name db`	Nombre del contenedor — Flask lo usará como hostname
`--network`	Conecta el contenedor a la red lab-net
`--mount`	Monta el volumen pgdata en la ruta de datos de Postgres
`-e POSTGRES_*`	Variables de entorno para configurar usuario y BD

VERIFICACIÓN

verificar PostgreSQL

```
docker ps
CONTAINER ID   IMAGE              STATUS          NAMES
xxxxxxxxxxxx   postgres:16-alpine Up 10 seconds   db  ✓
# Ver logs (esperar ~5 segundos)
docker logs db
...database system is ready to accept connections  ✓
# Conectarse directamente (opcional)
docker exec -it db psql -U admin -d appdb
appdb=# \q
```




```
PS C:\Users\josem> docker exec -it db psql -U admin -d appdb
psql (16.13)
Type "help" for help.

appdb=# \q

What's next:
    Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug db
    Learn more at https://docs.docker.com/go/debug-cli/
PS C:\Users\josem>
```

3 Crear la aplicación Flask

Código y Dockerfile

Creamos tres archivos en una carpeta mi-app/: la aplicación Flask, los requirements de Python y el Dockerfile para construir la imagen.

Crear carpeta y archivos

```
mkdir mi-app && cd mi-app
touch app.py requirements.txt Dockerfile
# Abre cada archivo con tu editor y pega el contenido de abajo  ✓
```

```
PS C:\Users\josem\mi-app2> New-item app.py, requirements.txt, Dockerfile


    Directory: C:\Users\josem\mi-app2


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----         5/4/2026   7:24 PM              0 app.py
-a----         5/4/2026   7:24 PM              0 requirements.txt
-a----         5/4/2026   7:24 PM              0 Dockerfile
```

■ app.py

app.py

```
from flask import Flask, jsonify
import psycopg2

app = Flask(__name__)

def get_conn():
        return psycopg2.connect(
            host="db",         # nombre del contenedor en la red
            dbname="appdb", user="admin", password="secret123"
        )

@app.route("/")
def home():
        conn = get_conn()
        cur = conn.cursor()
        cur.execute("SELECT version();")
        ver = cur.fetchone()[0]
        cur.close(); conn.close()
        return jsonify({"status": "ok", "pg_version": ver})

@app.route("/visitas")
def visitas():
        conn = get_conn()
        cur = conn.cursor()
        cur.execute("""
            CREATE TABLE IF NOT EXISTS visitas
                (id SERIAL, ts TIMESTAMPTZ DEFAULT now());
            INSERT INTO visitas DEFAULT VALUES;
            SELECT COUNT(*) FROM visitas;""")
        conn.commit()
        total = cur.fetchone()[0]
        cur.close(); conn.close()
        return jsonify({"total_visitas": total})

if __name__== "__main__":
        app.run(host="0.0.0.0", port=5000)
```

■ requirements.txt

requirements.txt

```
flask==3.0.3
psycopg2-binary==2.9.9
```

■ Dockerfile

Dockerfile

```
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 5000

CMD ["python", "app.py"]
```

4 Construir la imagen

docker build — crear imagen personalizada

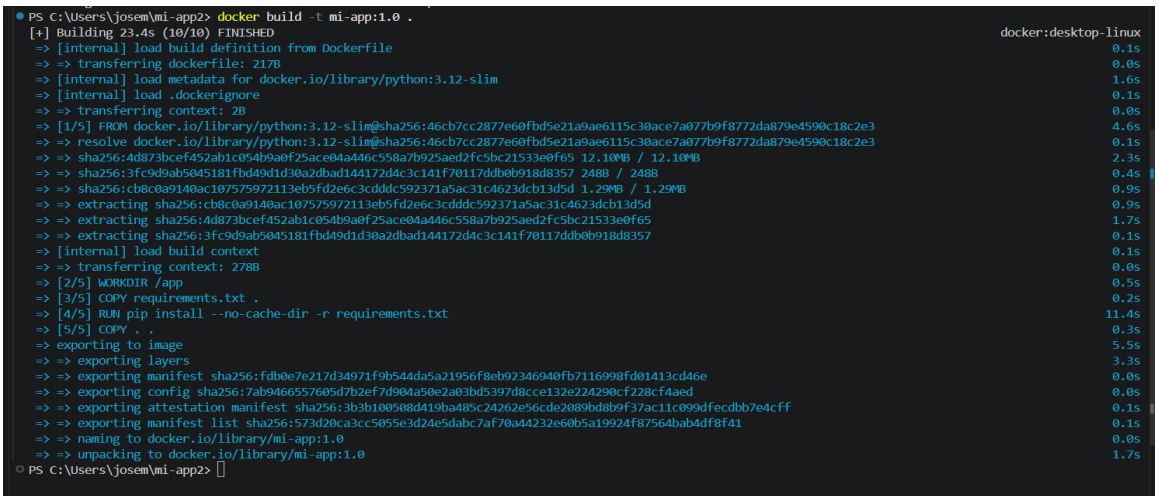
Construimos la imagen de nuestra app Flask desde el Dockerfile. Este proceso descarga la imagen base de Python, instala Flask y psycopg2, y empaqueta el código en una imagen reutilizable llamada mi-app:1.0.

docker build

```
# Desde dentro de la carpeta mi-app/
docker build -t mi-app:1.0 .

Step 1/6 : FROM python:3.12-slim
 ---> abc123ef...
Step 2/6 : WORKDIR /app
Step 3/6 : COPY requirements.txt .
Step 4/6 : RUN pip install --no-cache-dir -r requirements.txt
     Installing collected packages: flask, psycopg2-binary
Step 5/6 : COPY . .
Step 6/6 : CMD [python, app.py]
Successfully built def456ab...
Successfully tagged mi-app:1.0  ✓
```

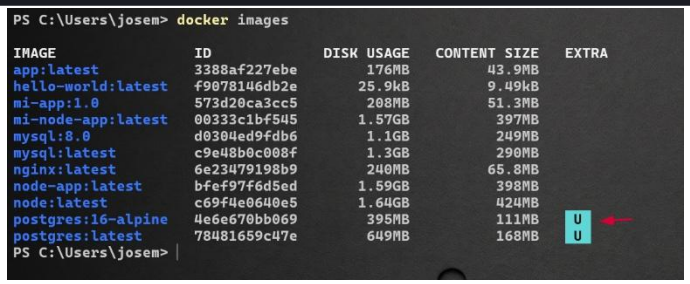
`-t mi-app:1.0`	Nombre y tag de la imagen resultante
`.`	Usa el directorio actual como contexto de build



VERIFICACIÓN

verificar imagen construida

```
docker images
REPOSITORY    TAG    IMAGE ID       CREATED          SIZE
mi-app        1.0    def456ab...    1 minute ago     ~180MB  ✓
```



5 Correr la app y probarla

Conectar todo y ejecutar

Lanzamos el contenedor de la app conectándolo a la misma red donde está el contenedor db. El flag -p expone el puerto 5000 del contenedor al puerto 5000 de tu máquina.

docker run — Flask app

```
docker run -d \
     --name app \
     --network lab-net \
     -p 5000:5000 \
     mi-app:1.0
9a8b7c6d5e4f3a2b1c0d9e8f7a6b5c4d3e2f1a0b
```

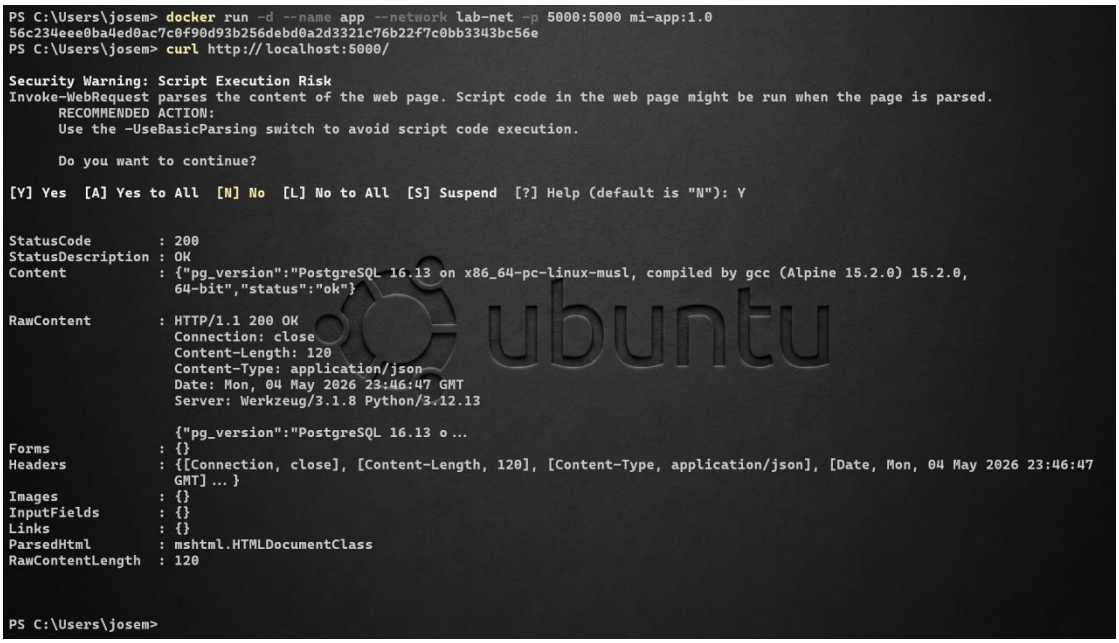
probar desde el host

```
# Probar conexión a la BD
curl http://localhost:5000/
{"pg_version": "PostgreSQL 16.x...", "status": "ok"}

# Contar visitas — crea tabla y registra cada hit
curl http://localhost:5000/visitas
{"total_visitas": 1}
curl http://localhost:5000/visitas
{"total_visitas": 2}

# También funciona en el navegador:
http://localhost:5000/visitas
```

✓ La app Flask se conecta a PostgreSQL usando host='db' porque ambos contenedores comparten la red lab-net y Docker resuelve el nombre automáticamente. Desde tu host accedes por localhost:5000 gracias al flag -p 5000:5000.



```
PS C:\Users\josem> curl http://localhost:5000/visitas

Security Warning: Script Execution Risk
Invoke-WebRequest parses the content of the web page. Script code in the web page might be run when the page is parsed.
      RECOMMENDED ACTION:
      Use the -UseBasicParsing switch to avoid script code execution.

      Do you want to continue?

[Y] Yes  [A] Yes to All  [N] No  [L] No to All  [S] Suspend  [?] Help (default is "N"): A


StatusCode        : 200
StatusDescription : OK
Content           : {"total_visitas":1}

RawContent        : HTTP/1.1 200 OK
                    Connection: close
                    Content-Length: 20
                    Content-Type: application/json
                    Date: Mon, 04 May 2026 23:48:10 GMT
                    Server: Werkzeug/3.1.8 Python/3.12.13

                    {"total_visitas":1}

Forms             : {}
Headers           : {[Connection, close], [Content-Length, 20], [Content-Type, application/json], [Date, Mon, 04 May 2026 23:48:10
                    GMT] ... }
Images            : {}
InputFields       : {}
Links             : {}
ParsedHtml        : mshtml.HTMLDocumentClass
RawContentLength  : 20


PS C:\Users\josem>
```

```
PS C:\Users\josem> curl http://localhost:5000/visitas


StatusCode        : 200
StatusDescription : OK
Content           : {"total_visitas":2}

RawContent        : HTTP/1.1 200 OK
                    Connection: close
                    Content-Length: 20
                    Content-Type: application/json
                    Date: Mon, 04 May 2026 23:48:53 GMT
                    Server: Werkzeug/3.1.8 Python/3.12.13

                    {"total_visitas":2}

Forms             : {}
Headers           : {[Connection, close], [Content-Length, 20], [Content
                    GMT] ... }
Images            : {}
InputFields       : {}
Links             : {}
ParsedHtml        : mshtml.HTMLDocumentClass
RawContentLength  : 20


PS C:\Users\josem>
```

VERIFICACIÓN

verificar ambos contenedores

```
docker ps
CONTAINER    IMAGE                STATUS     NAMES
xxx          mi-app:1.0           Up         app   ✓
yyy          postgres:16-alpine   Up         db    ✓
# Ver la red: ambos contenedores deben aparecer
docker network inspect lab-net
"Containers": { "app": {...}, "db": {...} }   ✓
```

```
PS C:\Users\josem> docker ps
CONTAINER ID   IMAGE                COMMAND                  CREATED          STATUS          PORTS                                       NAMES
56c234eee0ba   mi-app:1.0           "python app.py"          6 minutes ago    Up 5 minutes    0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp   app
a54f3ff629ac   postgres:16-alpine   "docker-entrypoint.s…"   35 minutes ago   Up 35 minutes   5432/tcp                                    db
0c0b1d45422d   postgres             "docker-entrypoint.s…"   7 hours ago      Up 7 hours      5432/tcp                                    mi_postgres
PS C:\Users\josem>
```

```
        "com.docker.network.enable_ipv6": "false"
    },
    "Labels": {},
    "Containers": {
        "56c234eee0ba4ed0ac7c0f90d93b256debd0a2d3321c76b22f7c0bb3343bc56e": {
            "Name": "app",
            "EndpointID": "c069d6f6c1bade7e43efc16d1b33708a409fc545d217ce9ee12a458b97f50e61",
            "MacAddress": "1a:7e:10:e7:8b:38",
            "IPv4Address": "172.20.0.3/16",
            "IPv6Address": ""
        },
        "a54f3ff629ac2795747cd15168618b8d30c48f97476255ed666c925d497d1534": {
            "Name": "db",
            "EndpointID": "3f130729db8af91a11c1611e3e87e2574107fbbd3cd018d0ecc3daae040881c1",
            "MacAddress": "da:1d:79:d5:83:44",
            "IPv4Address": "172.20.0.2/16",
            "IPv6Address": ""
        }
    },
    "Status": {
```

6 Probar persistencia del volumen

El volumen sobrevive al contenedor

Esta es la parte más importante: vamos a destruir el contenedor de PostgreSQL, recrearlo con el mismo volumen pgdata y verificar que los datos siguen ahí. Esto demuestra por qué los volúmenes son esenciales en producción.

probar persistencia paso a paso

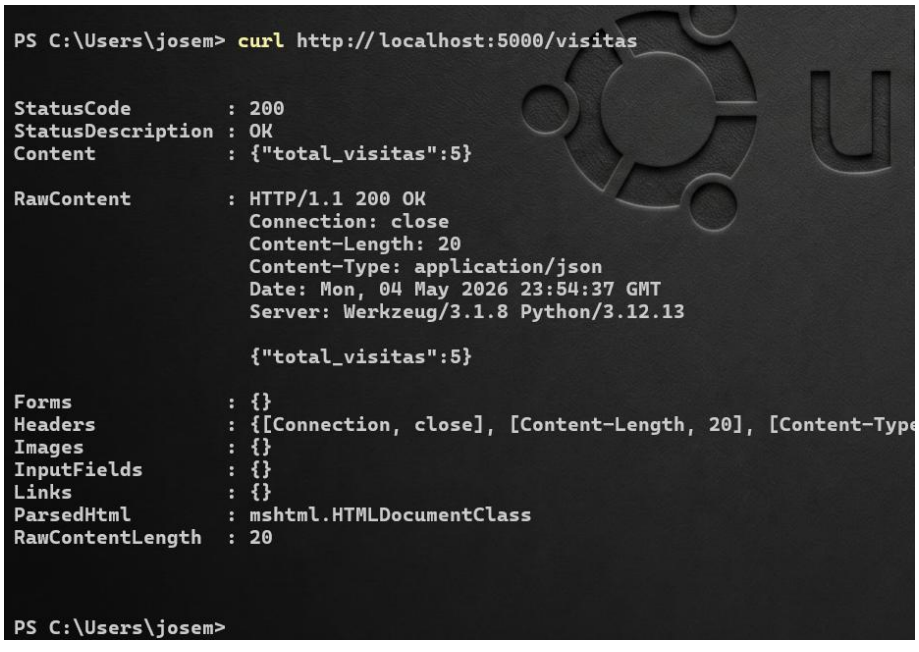
```
# 1. Acumula varias visitas
curl http://localhost:5000/visitas   # repite 4-5 veces
{"total_visitas": 5}

# 2. Destruye el contenedor de la BD (¡pero NO el volumen!)
docker stop db && docker rm db
db

# 3. Recrea el contenedor con el MISMO volumen pgdata
docker run -d \
     --name db --network lab-net \
     --mount source=pgdata,target=/var/lib/postgresql/data \
     -e POSTGRES_USER=admin -e POSTGRES_PASSWORD=secret123 \
     -e POSTGRES_DB=appdb postgres:16-alpine

# 4. Espera 5 segundos y consulta — los datos siguen ahí
curl http://localhost:5000/visitas
{"total_visitas": 5}  ← datos sobrevivieron al restart  ✓
```

■ Sin volumen, al borrar el contenedor se perderían todos los datos. Con --mount source=pgdata los archivos de Postgres viven en el host, independientes del ciclo de vida del contenedor.



```
PS C:\Users\josem> docker stop db
db
PS C:\Users\josem> docker rm db
db
PS C:\Users\josem> docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED          STATUS          PORTS                                       NAMES
56c234eee0ba   mi-app:1.0     "python app.py"          12 minutes ago   Up 12 minutes   0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp   app
0c0b1d45422d   postgres       "docker-entrypoint.s…"   7 hours ago      Up 7 hours      5432/tcp                                    mi_postgres
PS C:\Users\josem> docker run -d --name db --network lab-net --mount source=pgdata,target=/var/lib/postgresql/data -e POSTGRES_USER=admin -e POS
TGRES_PASSWORD=secret123 -e POSTGRES_DB=appdb postgres:16-alpine
bdc73e98100f80ef5273c49445a5e0d0c44de81f69f60c871c094cf990d89c8e
PS C:\Users\josem> curl http://localhost:5000/visitas


StatusCode        : 200
StatusDescription : OK
Content           : {"total_visitas":6}

RawContent        : HTTP/1.1 200 OK
                    Connection: close
                    Content-Length: 20
                    Content-Type: application/json
                    Date: Mon, 04 May 2026 23:57:57 GMT
                    Server: Werkzeug/3.1.8 Python/3.12.13

                    {"total_visitas":6}

Forms             : {}
Headers           : {[Connection, close], [Content-Length, 20], [Content-Type, application/json], [Date, Mon, 04 May 2026 23:57:57 GMT] ... }
Images            : {}
InputFields       : {}
Links             : {}
ParsedHtml        : mshtml.HTMLDocumentClass
RawContentLength  : 20


PS C:\Users\josem>
```

7 Limpiar el laboratorio

Borrar todo lo creado

Al terminar la práctica, limpiamos todos los recursos creados: contenedores, imagen, red y volumen.

limpiar todo

```
# Detener y borrar contenedores
docker stop app db
docker rm app db

# Borrar imagen construida
docker rmi mi-app:1.0

# Borrar red y volumen
docker network rm lab-net
docker volume rm pgdata

Laboratorio limpio ✓
```

VERIFICACIÓN FINAL

verificar limpieza completa

```
docker ps -a
CONTAINER ID   IMAGE   ...   (vacío) ✓
docker volume ls
DRIVER    VOLUME NAME   (vacío) ✓
docker network ls
bridge   host   none   (solo las default) ✓
```

✓ ¡Práctica completada! Dominaste: docker network create · docker volume create · docker build -t · docker run --network · docker run --mount · -p host:container

---

```
PS C:\Users\josem> docker stop app db
app
db
PS C:\Users\josem> docker rm app db
app
db
PS C:\Users\josem> docker rmi mi-app:1.0
Untagged: mi-app:1.0
Deleted: sha256:573d20ca3cc5055e3d24e5dabc7af70a44232e60b5a19924f87564bab4df8f41
PS C:\Users\josem> docker network rm lab-net
lab-net
PS C:\Users\josem> docker volume rm pgdata
pgdata
PS C:\Users\josem> docker ps -a
CONTAINER ID   IMAGE      COMMAND                  CREATED       STATUS       PORTS      NAMES
0c0b1d45422d   postgres   "docker-entrypoint.s…"   7 hours ago   Up 7 hours   5432/tcp   mi_postgres
PS C:\Users\josem> docker volume ls
DRIVER    VOLUME NAME
local     1d5cef87275614f0c75dd501a645b2709af82879815ac058958f91ffaa703e6f
local     2ac57fdd2d64c28b08e445036b9c2a6a273a6ff0a4dedbc90b719ae4e32409ca
local     6b4b675330b9b67db7829209e8fda3de33c4e188c26f0b63e70f52732e53f718
local     9891e58777489cf8f3272a15cf84d2b0f4ba8523d8af2737290d815cc1aa3784
local     c03c99d099a77cf16ff01d98c97800d174d179bc257dcefba50ef378bf5227b9
local     ce3a561a8481d1ccbb9371600ac1de65b2197fbe77567765298e4cdaab211af2
local     d4e6cd960002b19bd808ddbe8176131f05a54d6a63251c8ec7d3f6e42e3aac67
local     e3cb1e1aadf8b6a81ffd2873e41f16cc610c8071766e2b52c0b8ad201cc86f35
local     e0853b6dc2a47481a1fee45bf4969288871ed13965bd26634ad6cb32ccae3dfd
local     fb19c1fac8608784c7a67260e98e2ba28bcf4346ba155d314d165b7c09679cfd
local     mysql_datos
PS C:\Users\josem> docker network ls
NETWORK ID     NAME      DRIVER    SCOPE
775fbff3b143   app_red   bridge    local
52a2908eb214   bridge    bridge    local
e2f7fd92aaf3   host      host      local
a8296eacd3ed   mi_red    bridge    local
1ea06241dbb9   none      null      local
PS C:\Users\josem>
```

RESUMEN DE COMANDOS

Comando	Descripción
`docker network create --driver bridge NAME`	Crear red personalizada
`docker volume create NAME`	Crear volumen persistente
`docker build -t IMAGE:TAG .`	Construir imagen desde Dockerfile
`docker run --network NAME ...`	Conectar contenedor a una red
`docker run --mount source=VOL,target=PATH`	Montar volumen en contenedor
`docker run -p HOST:CONTAINER ...`	Exponer puerto al host
`docker network inspect NAME`	Ver contenedores en la red
`docker logs CONTAINER`	Ver logs del contenedor

`docker exec -it CONTAINER bash`	Terminal dentro del contenedor